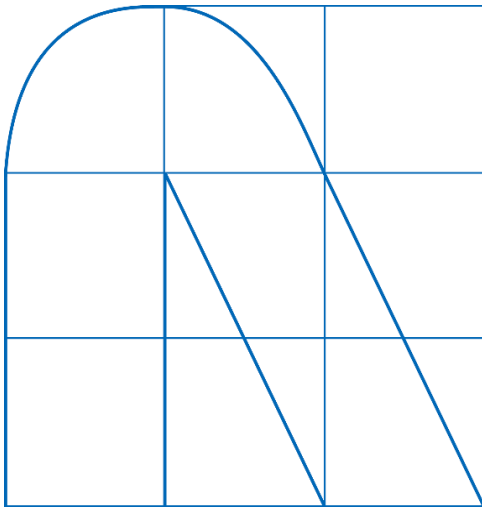


Info type: Confidential

Company: NTT Data Payment Services India

Info. owner: Product (PMG)



Transaction API (Non-Seamless)

[Version 6]

Contents

CONFIDENTIALITY DISCLAIMER.....	4
1. Document Information.....	4
2. Revision History.....	4
3. Introduction	6
4. Key Features.....	6
5. Payment Process	6
6. Transaction APIs	7
Process Flow	7
AUTH API.....	7
7.1 Create API Secret Key.....	8
Data Validation	9
7.2 Create a Bearer Token.....	9
cURL.....	9
Python	10
Java	11
Node.js	12
PHP	13
7.3 Error Handling.....	14
Error Response Format.....	14
7.4 Change under Endpoints	14
Current URL	15
Updated URL	15
8. Transaction API Details.....	15
Auth API Request Parameters	15
Auth API Request in JSON (Sample)	20
Auth API Request in Encrypted Format (Sample).....	21
Auth API Response Parameters	22
Auth API Response in Encrypted Format.....	22
Auth API Response in JSON Format.....	22
JAVASCRIPT	23
Java Script in JSON Format	23
Final Transaction Response.....	24
Response Parameters Table.....	24

Response in JSON Format.....	26
Response in Encrypted Format.....	28
Sample Decrypted Response.....	29
Signature Generation.....	33
Sample Request.....	33
Generated Signature	33
Hash Keys.....	33
Encryption Key.....	33
Signature Generation Java Code	33
AES Encryption Logic.....	35
AES Encryption Logic Code	35
9. UAT Environment Details.....	37
Test Credentials.....	38
Test Card Details	38
Test UPI ID	38
10. Sample Plain Request & Response (Single Account Settlement) for Multiple Payment Modes..	38
Plain Request In JSON	38
Net Banking Sample Response in JSON	39
Credit Card Sample Response in JSON.....	41
Debit Card Sample Response in JSON.....	42
UPI Sample Response in JSON	43
NEFT/RTGS Challan Sample Response in JSON	45
11. Sample Plain Request & Response (Split Settlement – Multi Product) for Multiple Payment Modes	46
Plain Request in JSON	46
Net Banking Sample Response in JSON	47
Credit Card Sample Response in JSON.....	48
Debit Card Sample Response in JSON.....	49
UPI Sample Response in JSON	50
NEFT / RTGS Challan Response in JSON.....	51

CONFIDENTIALITY DISCLAIMER

The information included in this document is confidential information relating to the business of NTT Data Payment Services, India (NDPS). It is being presented to you based on the understanding that it will not be used for any reason other than consideration of a commercial relationship with NDPS and, will not be used in connection with any decision to trade in securities of NDPS. Please be advised that any disclosure of the information contained in this document/presentation to any other person, or any use of this information in connection with the trading of NDPS securities, may be a violation.

This document, or any part of it, may not be reproduced, copied, circulated and/or distributed nor quoted without prior written approval from NDPS.

1. Document Information

Document Attributes	Information
Document Name	Transaction API (Non-Seamless)
Document Version	V6
Owner	PMG
Author	P&E
Approved	

2. Revision History

Version	Author	Description of Version	Date	Reviewed By
1.0	Nikhil Dayma	Transaction API (Non-Seamless)	02-08-2022	
2.0	Nikhil Dayma	Process Flow Diagram Change	04-08-2022	
3.0	Ashesh Joshi	Changes in the description of the request table	04-03-2025	
4.0	Ashesh Joshi	Sample Request / Response JSON added for multiple payment modes	15-05-2025	

Version	Author	Description of Version	Date	Reviewed By
5.0	Aditi Jain	- Updated company logo and revision history added. - Content re-structuring & sequencing. - Process Flow diagrams updated in the Transaction API. - Encrypted response for production removed under transaction API. - Sample Plain Request & Response (Single Account Settlement) for Multiple Payment Modes heading change. - Sample Plain Request & Response (Split Settlement – Multi Product) for Multiple Payment Modes heading change.	18-06-2026	Nital Kapadia
6.0	Mukul Vishwakarma	Introduction to API Secret Key and Modification in Endpoints	07-08-2026	

3. Introduction

Transaction API (Non-Seamless) facilitates the payment process from the initiation of the transaction to the authorization of the payment by the customer on the Payment Page. The transaction-related data is exchanged server-to-server directly via RESTful APIs. This API acts as a method to obtain the payment pop-up post request authorization and get it displayed on the checkout page.

By integrating this API into your system, you can offer your customers a wide range of payment options. It is optimized with device screen size and helps customers to pay easily and securely over web or on mobile. It supports payment options such as Credit Cards, Debit Cards, Net Banking, UPI, Wallets & NEFT/RTGS.

It also provides a retry option for reducing customer drop-offs in case of transaction failure. Customer can retry the transaction by selecting alternate payment options.

4. Key Features

- Easy integration
- Optimized UI
- PCI-DSS complaint
- User-friendly Customer UI
- Provides retry option, reducing customer drop

5. Payment Process

Below is the end-to-end payment process flow from –

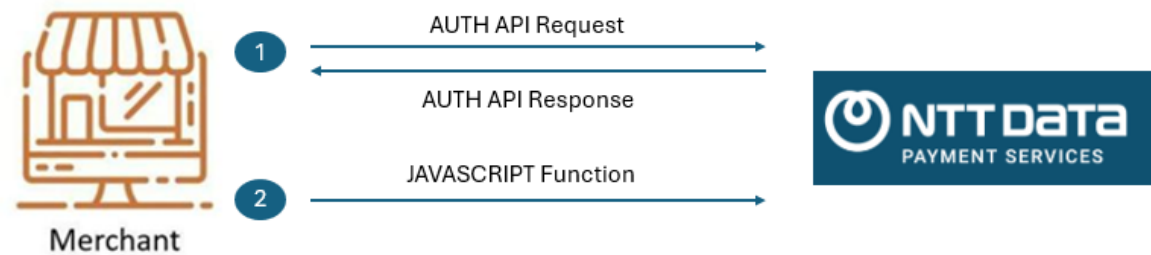
1. Once the customer is at merchant's checkout page, the merchant calls the NDPS' Rest API for payment authentication.
2. Rest API request needs to be encrypted using the AES-512 algorithm.
3. Token ID will be generated by NDPS and sent in response to the AUTH API request.
4. The response will also be encrypted using the AES-512 algorithm.
5. The merchant is required to decrypt this response and thereby acquire the token ID.
6. This token ID is to be used for calling the NDPS' JavaScript function which will then open the payment page.
7. End customer can edit the mobile number and email ID which was sent by the merchant in the Rest API request and then continue to make the payment.
8. Payment can be done by the customer using any of UPI, Net Banking, Credit card, Debit card, NEFT/RTGS or Wallet as the payment option.
9. Once the payment is done, NDPS will send the response to the merchant entailing the status of the transaction (success, failed or timeout).
10. In case if the token ID is not received by the merchant or the response is not received in the specified time-period then the error message will be shown to the merchant i.e., "Endpoint request timed out".
11. Such errors are to be handled explicitly by the merchant and Rest API request has to be reinitiated.

6. Transaction APIs

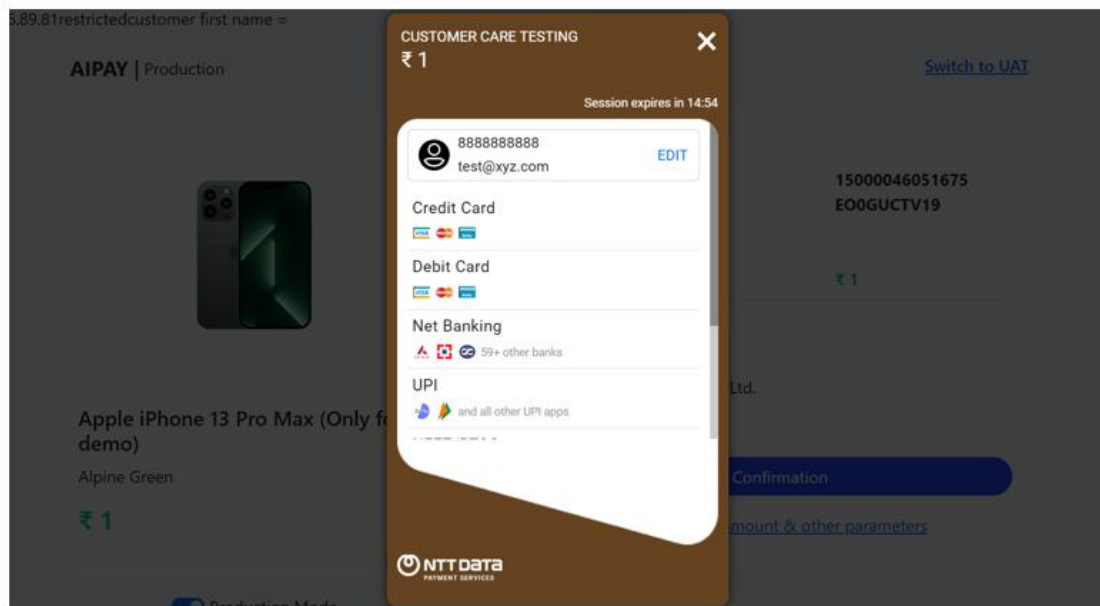
Process Flow

Two API calls need to be triggered simultaneously for the exhibition of the payment page on merchant's website. The details of the same can be found below -

A: MERCHANT REQUESTS



B: PAYMENT PAGE EXHIBITS



AUTH API

1. Merchant is required to initiate the Rest API to initiate payment link creation.
2. Authentication API used to generate the Token ID.
3. Post this, the Token ID will be used to call the JavaScript function to then land on the Payment pop-
4. Token ID generation will be server-to-server using POST method.
5. The request and response shall be encrypted and decrypted using AES -512 encryption algorithm and a signature generated using HMACSHA-512 algorithm.

API Secret Key (This needs to be implemented for additional authorization on both the platforms i.e. UAT and Production)

Note: API Secret Key for UAT and Production will be different. *For an instance shown in below table.*

API Secret Key	msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v
----------------	--

Process to generate API Secret Key is instructed below.

7.1 Create API Secret Key

Note: Once the merchant is onboarded, a new API secret key will be shared along with other existing keys (Request Key /Response Key / Request Hash Key / Response Hash Key)

If new API secret key needs to be generated, merchant needs to navigate to **API Credentials** module and generate new key. The key is displayed once; the merchant needs to copy and store it securely. The key cannot be shown again after the merchant leaves the page.

Key Generation

1. Log into Merchant Console with 2MFA
2. Navigate to "**API Credentials**"
3. Click "**Generate API Secret Key**"
4. New API Secret key is displayed once — copy and store it securely
5. The key cannot be shown again after the merchant leave the page

Key Rotation

Steps to rotate the key:

1. Log into Merchant Console with 2MFA
2. Click "**Rotate API Secret Key**"
3. New key is displayed once — copy and store it securely
4. Old key remains valid for 48 hours (grace period)
5. After 48 hours, old key stops working automatically
6. Merchant will receive an email confirmation with the grace period deadline

Key Revocation (Emergency)

If The merchant suspect the key is compromised:

1. Log into Console → Click "**Revoke API Secret Key**"
2. Key is immediately invalidated (no 48h grace period)
3. Generate a new key to restore API access

Data Validation

API Secret Key	Format	Type	Example
Total Length → 48 Characters	Always starts with → msk_XXXXXXXXXXXXXXXXXXXXXXX	Mix of uppercase, lowercase, and digits	msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v

7.2 Create a Bearer Token

To create a Bearer Token, below format is used.

Format	merchantId:apiSecretKey
--------	-------------------------

After this, the data is converted to Base64 format.

Once the data is formatted, it can be used to send in the Bearer token:

Authorization: Bearer <token>

Steps to follow:

- Step 1 — Generate Your API Secret Key
 - Log into Merchant Console (2MFA) → Generate API Secret Key → Store securely.
- Step 2 — Implement Bearer Token
 - Store merchantId and apiSecretKey securely on your server
 - Build credential string: merchantId:apiSecretKey
 - Base64-encode it
 - Add Authorization: Bearer <encoded> header

cURL

bash

```
#!/bin/bash

MERCH_ID="546852"

API_SECRET_KEY="msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v"

ENC_DATA="0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C..."

# Set the API path - replace with any affected endpoint from Section 5
API_PATH="/ots/v2/payment/status"

# Build Bearer token
TOKEN=$(echo -n "${MERCH_ID}:${API_SECRET_KEY}" | base64)
```

```
# Make request

curl -X POST "${BASE_URL}${API_PATH}?merchId=${MERCH_ID}&encData=${ENC_DATA}" \

-H "Authorization: Bearer ${TOKEN}"
```

Python

python

```
import base64

import requests

MERCH_ID = "546852"

API_SECRET_KEY = "msk_xk9mP2qR7vT5wZaB3cD4eF6gH8jKlmN0pQ9rS2tU4v"

# Set the API path - replace with any affected endpoint from Section 5
API_PATH = "/ots/v2/payment/status"

enc_data = "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C..."

# Build Bearer token

credentials = f"{MERCH_ID}:{API_SECRET_KEY}"

token = base64.b64encode(credentials.encode("utf-8")).decode("utf-8")

headers = {"Authorization": f"Bearer {token}"}

response = requests.post(

    f"{BASE_URL}{API_PATH}",

    params={"merchId": MERCH_ID, "encData": enc_data},

    headers=headers,

)

print(response.status_code, response.json())
```

Java

java

```
import java.net.HttpURLConnection;

import java.net.URL;

import java.net.URLEncoder;

import java.nio.charset.StandardCharsets;

import java.util.Base64;

public class OtsApiClient {

    private static final String MERCH_ID = "546852";

    private static final String API_SECRET_KEY =

        "msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jKlMN0pQ9rS2tU4v";

    public static void main(String[] args) throws Exception {

        // Set the API path – replace with any affected endpoint from Section 5

        String apiPath = "/ots/v2/payment/status";

        String encData =

            "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...";

        // Build Bearer token

        String credentials = MERCH_ID + ":" + API_SECRET_KEY;

        String token = Base64.getEncoder().encodeToString(

            credentials.getBytes(StandardCharsets.UTF_8));

        String urlStr = BASE_URL + apiPath

            + "?merchId=" + URLEncoder.encode(MERCH_ID, "UTF-8")

            + "&encData=" + URLEncoder.encode(encData, "UTF-8");

        URL url = new URL(urlStr);
```

```

        HttpURLConnection conn = (HttpURLConnection) url.openConnection();

        conn.setRequestMethod("POST");

        conn.setRequestProperty("Authorization", "Bearer " + token);

        System.out.println("Response Code: " + conn.getResponseCode());

    }
}

```

Node.js

javascript

```

const https = require('https');

const MERCH_ID = '546852';

const API_SECRET_KEY = 'msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jKlMnOpQ9rS2tU4v';

// Set the API path - replace with any affected endpoint from Section 5
const API_PATH = '/ots/v2/payment/status';

const encData = '0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...';

// Build Bearer token
const token = Buffer.from(`${MERCH_ID}:${API_SECRET_KEY}`).toString('base64');

const queryParams =

    `merchId=${encodeURIComponent(MERCH_ID)}&encData=${encodeURIComponent(encData)}`;

const options = {

    hostname: BASE_HOST,

    path: `${API_PATH}?${queryParams}`,

    method: 'POST',

```

```

    headers: { 'Authorization': `Bearer ${token}` },
  };

const req = https.request(options, (res) => {

  let data = '';

  res.on('data', (chunk) => { data += chunk; });

  res.on('end', () => { console.log(res.statusCode, data); });

});

req.end();

```

PHP

php

```

<?php

$merchId      = "546852";

$apiSecretKey = "msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v";

// Set the API path - replace with any affected endpoint from Section 5
$apiPath      = "/ots/v2/payment/status";

$encData      = "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...";

// Build Bearer token
$token = base64_encode("${merchId}:${apiSecretKey}");

$url = "${baseUrl}${apiPath}?merchId=" . urlencode($merchId)
      . "&encData=" . urlencode($encData);

$ch = curl_init($url);

curl_setopt($ch, CURLOPT_POST, true);

curl_setopt($ch, CURLOPT_HTTPHEADER, ["Authorization: Bearer ${token}"]);

```

```

curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);

$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);

curl_close($ch);

echo "HTTP {$httpCode}: {$response}\n";

```

7.3 Error Handling

HTTP Code	Error Message	Cause	Resolution
401 Unauthorized	Missing authorization	Authorization header not present	Add Authorization: Bearer <token> header
401 Unauthorized	Invalid token format	Not valid Base64 or missing ':' separator	Verify Base64 encoding of merchantId:apiSecretKey
401 Unauthorized	Invalid credentials	merchantId or apiSecretKey does not match	Check correct key for correct environment
401 Unauthorized	Key expired	API Secret Key expired (past 48h grace)	Generate new key from Console
403 Forbidden	Merchant suspended	Account suspended	Contact NDPSI support

Error Response Format

json

```

{
  "message": "Unauthorized"
}

```

Note: Other than the above addition, all the existing error code and error message remains same.

7.4 Change under Endpoints

To authenticate using the new API Secret Key, merchants must update the API endpoint URL.

Example:**Current URL**

https:// payment.atomtech.in/ots/aipay/auth

Updated URL

https:// payment.atomtech.in/otsv2/aipay/auth

8. Transaction API Details

Auth API Request Parameters

Parent JSON field Name	Parameter Name	Data Type	Max Length	M / O / C	Sample Value	Static/ Dynamic	Description
merchDetails	merchId	Number	10	M	330703	Static	NDPS Merchant MID
	userId	String	45	O	"123"	Dynamic	Customer's User ID
	password	String	80	M	"cf7b79f6"	Static	Transaction Password provided by NDPS
	merchTxnId	String	50	M	"Test123450"	Dynamic	Unique Transaction Reference No. provided by Merchant
	merchTxnDate	Datetime	19	M	"2023-07-13 20:46:00"	Dynamic	Transaction date in format -> YYYY-MM-DD hh:mm:ss
payDetails	amount	Number	12, 2	M	1.00	Dynamic	Total transaction amount (10 digits prior to the decimal)
	product	String	50	M	"NCA"	Dynamic	Unique value provided by NDPS which pertains to

							merchant settlement account number
	custAccNo	String	45	C	"213232323"	Dynamic	End customer's account number (Only mandatory for TPV transaction)
	txnCurrency	String	5	M	"INR"	Static	Transaction currency type
custDetails	custEmail	String	100	M	testuser@atomtech.in	Dynamic	End customer's email ID
	custMobile	String	20	M	"9753864210"	Dynamic	End customer's mobile number
extras	platformOS*	String	50	CM	-	Dynamic	This is only applicable when subChannel is UP. Android / iOS.
	Udf1	String	45	O	-	Dynamic	User Definer Parameter 1
	Udf2	String	45	O	-	Dynamic	User Definer Parameter 2
	Udf3	String	45	O	-	Dynamic	User Definer Parameter 3
	Udf4	String	45	O	-	Dynamic	User Definer Parameter 4
	Udf5	String	45	O	-	Dynamic	User Definer Parameter 5
	Udf6	String	45	O	-	Dynamic	User Definer

							Parameter 6
	Udf7	String	45	O	-	Dynamic	User Definer Parameter 7
	Udf8	String	45	O	-	Dynamic	User Definer Parameter 8
	Udf9	String	45	O	-	Dynamic	User Definer Parameter 9
	Udf10	String	45	O	-	Dynamic	User Definer Parameter 10
	challanDetails*	String	400	O	-	Dynamic	Additional Challan details to be printed under this parameter
payModeSpecificData	subChannel	String	40	O	"NB"	Dynamic	Use to exhibit specific payment mode
prodDetails	prodName	String []	50	O	"NCA"	Dynamic	Product name registered with NDPS (multiple product can be passed as an array)
	prodAmount	Number	12,2	O	1.00	Dynamic	Product Specific Amount (10 digits prior to the decimal)

***Note:** the possible values for platform would be as follows: iOS, Android. These values can be derived using user agent parameters.

What is a User Agent?

A User Agent (UA) is a string sent by a browser or app to a server that identifies:

1. Browser name & version
2. Operating System

3. Device type (mobile/desktop/tablet)
4. Rendering engine (WebKit, Blink, Gecko, etc.)

It is sent in the HTTP header:

User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36

Real User Agent Examples:

Chrome on Mac

Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)

AppleWebKit/537.36 (KHTML, like Gecko)

Chrome/144.0.0.0 Safari/537.36

Safari on iPhone

Mozilla/5.0 (iPhone; CPU iPhone OS 17_0 like Mac OS X)

AppleWebKit/605.1.15 (KHTML, like Gecko)

Version/17.0 Mobile/15E148 Safari/604.

Sample Request for Challan

*** ChallanDetails** - As mentioned in the table above, a new parameter has been added under the Extras section. This is a custom requirement from the merchant, which mandates that additional challan details be printed on the challan.

Note: The response for the same will be printed directly on the Challan receipt. Response table remains unchanged.

```
{
  "payInstrument": {
    "headDetails": {
      "version": "OTSv1.1",
      "api": "AUTH",
      "platform": "FLASH"
    },
    "merchDetails": {
      "merchId": "317157",
      "userId": "",
      "password": "Test@123",
      "merchTxnId": "CDXRV3I68M",
      "merchTxnDate": "2025-06-06 08:32:40"
    },
    "payDetails": {
      "amount": "11",
      "product": "NSE",
      "custAccNo": "213232323",
      "txnCurrency": "INR",
      "clientCode": ""
    }
  }
}
```

```

},
"custDetails": {
  "custFirstName": "",
  "custEmail": "testsa888@gmail.com",
  "custMobile": "8989889800"
},
"extras": {
  "udf1": "",
  "udf2": "",
  "udf3": "",
  "udf4": "",
  "udf5": "",
  "udf6": "",
  "udf7": "",
  "udf8": "",
  "udf9": "",
  "udf10": "",
  "challanDetails":
"challanNo=12345|propertyCode=code|scheme=naviMumbai|phoneNumberOfAllotee=8007100190|challanType=gst"
}
}
}

```

Below are the possible values for Parameter 'subChannel' –

Field Value	Description
NB	In case the merchant wishes to show only Net Banking as payment option to customer.
CC	In case the merchant wishes to show only Credit Card as payment option to customer.
DC	In case the merchant wishes to show only Debit Card as payment option to customer.
MW	In case the merchant wishes to show only Wallet as payment option to customer.
PP	In case the merchant wishes to show only PhonePe as payment option to customer.
PW	In case the merchant wishes to show only Paytm Wallet as payment option to customer.
EM	In case the merchant wishes to show only EMI as payment option to customer.
NR	In case the merchant wishes to show only Challan as payment option to customer.
BQ	In case the merchant wishes to show only BharatQR as payment option to customer.
UP	In case the merchant wishes to show only Unified Payment Interface (UPI) as payment option to customer.

Specifications for the Amount to be split to multiple Accounts:

Merchant can specify multiple products in 'prodDetails' if the same is pre-configured. The amounts across those products can be variable, but the total amount across all the products should be equal to that paid by the end-user.

Process flow diagram:



If there are various products across which the payment is to be distributed, the 'product' parameter shall be replaced by the '**prodDetails**' parent JSON field as depicted in the example below –

The "product": "NSE", shall be replaced by: -

```
"prodDetails": [  
{  
  "prodName": "NSE",  
  "prodAmount": 11.00  
},  
{  
  "prodName": "BSE",  
  "prodAmount": 12.00  
}  
],
```

In such request, the sum of all 'prodAmount' should be equal to the amount to be paid.

Auth API Request in JSON (Sample)

```
{  
  "payInstrument": {  
    "headDetails": {  
      "version": "OTSv1.1",  
      "api": "AUTH",  
      "platform": "FLASH"  
    },  
    "merchDetails": {  
      "merchId": "317157",  
      "userId": "",  
      "password": "Test@123",  
      "merchTxnId": "019MC6VE42",  
      "merchTxnDate": "2024-12-19 06:20:42"  
    },  
    "payDetails": {  
      "amount": "319",  
      "product": "NSE",  
      "custAccNo": "213232323",  
      "txnCurrency": "INR",  
      "clientCode": ""  
    }  
  }  
}
```

```

"custDetails": {
  "custFirstName": "Ravina Solse",
  "custEmail": "hrd-mum789@ai.in",
  "custMobile": "09754564348",
  "platformOS": "iOS"
},
"extras": {
  "udf1": "One",
  "udf2": "Two",
  "udf3": "Three",
  "udf4": "Four",
  "udf5": "Five",
  "udf6": "Six",
  "udf7": "Seven",
  "udf8": "Eight",
  "udf9": "Nine",
  "udf10": "Ten"
}
},
"payModeSpecificData": {
  "subChannel": "DC"
}
}

```

Auth API Request in Encrypted Format (Sample)

Note:

- Merchant needs to encrypt the request using the AES -512 encryption algorithm.
- It is mandatory to pass the 'encData' and 'merchId' in the REST API call.

Sample Encrypted Request

```

FFC44EED0A1AE8C8FFDA9352895059AD02744AD2AA34BBDC0BDCD2680CF66E19873972E4F7571536A0D
09A9FEC71A544E3489A49F2AB899A7F9C0A682419BF362DD2D89047F299B8E03DCC6BE49F963F852C25E
B2ECEC54EFB4D59777831AF03A8BDC61D609C5CBF7D9BDF797451A4F07BB04323673A6E04E7F755243B1
13E06E3B48B42230FE19A2C8D08EAB3D34BEF53D0D3EF9573DAA6D004280E71983B83E77824C5D0BDC0
911C913DAD179B4E75FB2260EA2CCAB12098ED5E2F53A357D11C6AB718076F73BFF167CB6D3D61B5821
4AE84EA0AF69D4A25F2D834767288957F20A3C6D92EE38B926E697A97BA2BE9EBBC53364B0E6864F17A4
784055B2FFFFEDAA8F0D18C7ECBE5EA82D798558CA1C44B27882790A8184DA533F2149989D2D031A3657
11FF7D796F59DD2BD17E29B5D7734BD46D7A7106F47D381E79FAD0A5DD02BE3A5CEFA4BB4360294710
D8CF7D9A458567E73DD2DCDD3F262A7DAF90CE8F574F1726651AB0C4DD6D3EC5C6EACFA393496A47CB
0ACA0FD8299E817400E3A84331AE307F468805B0257FC2D0192DDABAB7C7E1EDDA9D6DB1EE07F78A73B
F2E2DACE2FD5CDB70A6B86F3FA77D021D6A9030EAF9B5298B8418E211397DBC27ABF66AC004B7B13D8A
C9C9F7057505283AD8C05D8DB10D321370FACCB1C1B157B2CE7E0A8678793B3D57892E0F4662FA6FA2F8
E72D3C6C17EA4CA49853BE79037ABD022ADA162DAD4E5DE2F0349A9FB4D1F375DB637C6303CCEA7AEA
E9A43E25DA326E074B09C51AE6F3298727C6FBC18D8E92636F6FBC9F1D90607988CE8B5E262B053890B6
2029B7E3E75970F9BC428D388CF990B6A65B588B1A7A8138C991471344785A414B70F97BE77D04ED813F5
696DA97DB384208DE9F2151BFCD750A922F773B750A612113A3267732DD5

```

Auth API Response Parameters

Parent JSON field Name	Parameter Name	Data Type	Max Length	Sample Value	Static / Dynamic	Description
encData	atomTokenId	Number	15	15000008670129	Dynamic	Transaction Specific Token ID Generated by NDPS
	txnMessage	String	20	"SUCCESS"	Dynamic	Status Message
	txnStatusCode	String	10	"OTS0000"	Dynamic	Status
	txnDescription	String	100	"ATOM TOKEN ID HAS BEEN GENERATED txnDescription string 100 Dynamic Status Description SUCCESSFULLY"	Dynamic	Status Description

Auth API Response in Encrypted Format

merchId=330703&encData=67D1988C3CB9D7BAE404AD775CAD04C7CF51F4097AD369F73CD223B5A370328A53BED3977D2EB34EFE0692D8001950B37FD22D1A79992BBB7B7DA5D51509FEC08BA160760758718B7CB1679C61B8C3F9BCD2276DD306FDBBBFB26DAC7CF13D0E2C638E774D78BC418CF36B383371FE7ECB945AB8A505843DAF91861FD2D961F3DA42791E77C364A350BB5D48BAF864B53E24F07E2E986E7AF0AD1B33C73779C1E8A3EDC36C62ED3156B1B7F32C6C27D9

Auth API Response in JSON Format

```
{
  "atomTokenId": 15000000033303,
  "responseDetails": {
    "txnStatusCode": "OTS0000",
    "txnMessage": "SUCCESS",
    "txnDescription": "ATOM TOKEN ID HAS BEEN GENERATED SUCCESSFULLY"
  }
}
```

Key Points:

- Merchant needs to decrypt the response received using the AES-512 decryption algorithm.
- Post decryption of the 'atomTokenId' passed by NDPS in the response will be received by the merchant.

Possible status code values and messages that will be posted in response while generating the Atom Token Id are mentioned below:

Sr. No	txnStatus	txnMessage	txnDescription
1	OTS0000	SUCCESS	ATOM TOKEN ID HAS BEEN GENERATED
2	OTS0451	FAILED	INVALID MERCHANT INFORMATION
3		FAILED	AUTH SERVICE FAILED
4		FAILED	VALIDATION FAILED
5		FAILED	TOKEN GENERATION FAILED

JAVASCRIPT

- Once the 'atomTokenId' is received by the merchant, the JavaScript function needs to be called by the merchant.
- This JavaScript function can be called using the button click
- Mandatory JavaScript CDN link for UAT: <https://pgtest.atomtech.in/staticdata/ots/js/atomcheckout.js>
- Mandatory JavaScript CDN link for Production: <https://psa.atomtech.in/staticdata/ots/js/atomcheckout.js>

Java Script in JSON Format

```
<script > function openPay()
{
const options =
{
"atomTokenId": "atomTokenId",
"merchId": "8952",
"custEmail": "testuser@ndps.in",
"custMobile": "8888888888",
"returnUrl": "http://your-response-url/response.php"
}
let atom = new AtomPaynetz(options, 'uat');
}
</script>
```

Final Transaction Response

- Once the transaction is completed by the end user the final transaction response will be sent to the merchant by NDPS
- Transaction response will provide the details such as the payment mode used, Bank Name, Bank Transaction ID, Masked Card Number etc.
- Merchant can capture the details and accordingly show the response to the end customer.

Response Parameters Table

All the below parameters are enclosed in the Parent JSON Header – ‘payInstrument’.

Parent JSON field Name	Parent JSON field Name	Parameter Name	Data Type	Length	Description	Mandatory / Optional
merchDetails		merchId	number	15	Merchant Unique Reference Number shared by NDPS	M
		merchTxnId	string	50	Transaction Id pertaining to merchant	M
		mccCode	string	7	Master Category Code provided by NDPS	M
		merchTxnDate	datetime	19	Transaction Date	M
payDetails		atomTxnID	number	25	NDPS Transaction ID	M
	prodDetails []	prodName	string	50	Name of the Product	M
		prodAmount	number	12,2	Product Specific Amount	M
		Amount	number	12,2	Amount	M
		surchargeAmount	number	12,2	Not to be passed	M
		totalAmount	number	12,2	Total Amount	M
		custAccNo	String	45	Bank Account Number of end user	C (only for TPV)
		txnInitDate	datetime	19	NDPS Txn Initiated Date	M
		txnCompleteDate	datetime	19	Transaction Complete Date	M
		custAcclfsc	String	45	IFSC of end user	
		txnCurrency	String	5	Currency used for transaction	M
		Signature	String	256	Signature generated	M

		Remarks	String	20	Remark passed by Merchant	M
		clientCode	String	45	Unique Client Code passed by Merchant	M
custDetails		custFirstName	String	50	Name received in response from Issuing Bank	M
		custLastName	String	50	Last Name of User	M
		custEmail	Email	100	Email of end user	M
		custMobile	String	45	Mobile of End User	M
	billingInfo	custAddr1	String	200	Customer Address Line 1	M
		custAddr2	String	200	Customer Address Line 1	M
		custCountry	String	45	Country	M
		custCity	String	30	City	M
		custState	String	30	State	M
		custZipCode	String	10	Zipcode	M
payModeSpecificData	bankDetails	Subchannel []	String	40	Sub Channel Id	M
		otsBankId	String	10	Bank ID stored at Bank's end	M
		otsBankName	String	60	Sponsor Bank Name	M
		cardType	String	20	CC/DC	CC-MDC-M
		cardMaskNumber	String	20	Masked Card Number	CC-MDC-M
		qrString	String	200	QR String	O
		bankTxnId	String	30	Bank Reference Id	M
		authId	String	10	Bank authId	CC-MDC-M UP-M
		scheme	String	15	Scheme Name (VISA, MASTER etc.)	CC-MDC-M
		Message	String	30	Success/Failure Message/ Pending	M

responseDetails		Description	String	50	Description of Transaction	M
		statusCode	String	30	Status Code of Transaction	M
extras		udf1	string	45	User Defined Parameter 1	O
		udf2	string	45	User Define Parameter 2	O
		udf3	string	45	User Define Parameter 3	O
		udf4	string	45	User Define Parameter 4	O
		udf5	string	45	User Define Parameter 5	O
		udf6	string	255	User Define Parameter 6	O
		udf7	string	255	User Define Parameter 7	O
		udf8	string	255	User Define Parameter 8	O
		udf9	string	255	User Define Parameter 9	O
		udf10	string	255	User Define Parameter 10	O

Message Description:

In response message states pending and gets redirected in cases wherein response is awaited from the bank's side in case of dual verification. OTS0551 is the message code which gets reflected in case the message is received from the status is awaited from the bank's side.

Response in JSON Format

```
{
  "payInstrument": {
    "merchDetails": {
      "merchId": "317157",
      "merchTxnId": "019MC6VE42",
      "merchTxnDate": "2024-12-19T11:56:37"
    },
    "payDetails": {
```

```
"atomTxnId": 11000000615892,
"prodDetails": [
  {
    "prodName": "NSE",
    "prodAmount": 319
  }
],
"amount": 319,
"surchargeAmount": 17.67,
"totalAmount": 336.67,
"custAccNo": "213232323",
"client Code": "1234",
"txnCurrency": "INR",
"signature":
"652d8df185e4073aa000f51ca2667f0e00458c4470061b1703e42981bb28887fb36145fb60b3cb3b3ab8f26d4
0c5a7b8019fac462101c436ee1c8e2e0731c339",
"txnInitDate": "2024-12-19 11:56:41",
"txnCompleteDate": "2024-12-19 11:57:21"
},
"payModeSpecificData": {
  "subChannel": [
    "DC"
  ],
  "bankDetails": {
    "otsBankId": 2,
    "bankTxnId": "00110000006158921119",
    "authId": "00000",
    "otsBankName": "Hdfc Bank",
    "cardType": "DC",
    "cardMaskNumber": "XXXXXXXXXXXX4444",
    "scheme": "VISA"
  }
},
"extras": {
  "udf1": "One",
```

```

"udf2": "Two",
"udf3": "Three",
"udf4": "Four",
"udf5": "Five"
},
"custDetails": {
  "custFirstName": "Ravina Solse",
  "custEmail": "hrd-mum789@ai.in",
  "custMobile": "09754564348",
  "billingInfo": {}
},
"responseDetails": {
  "statusCode": "OTS0000",
  "message": "SUCCESS",
  "description": "SUCCESS"
}
}
}

```

Response in Encrypted Format

- Final Transaction response will be sent by NDPS in the encrypted format.
- Merchant needs to decrypt the response using AES-512 decryption algorithm.

```

merchId=8952&encData=0CD63904F407DC4D24167F8D8F2BEE997C80D4536815B71C1EDDB8FDCBACE8
B9F6D72174CC18263D88AD7728C821A1F7B789A3F6C8EE43C28933A6748D3239346F4AAEC04AFFD768
7FB31FFDA9AAF0F64A2911990F315B425F11F1196EAF59E9471644C321A0050BF64C72DA0EFE6A11D14F
4AA3890267552FA4E3FA84B21275150F7E98CCF84BD145DCB6F00EF7494FAEE9D23852507F46517B67D
8F9680941BF887A761D9B26879B2A369E9B6CEF835DD51DEE86982255CCB98A8C9D8C3662FB87AF859
B9E0DF167D7D48E89BE6896226C5D8AED6B2CA1986325D030B646ADAD4C3944A8B851D3D3448C7F0C
D37DB8E466A51AFFD43E38307E2ECA29675614AC4DAA072CB6595974E4E1EEEE70F031B7BA3C5FDF2D
4F599854B658B9D9D2DAA3FEAA67B534BFADE98FB5D123BB8FDB563F840B7B28C71054CCA358F50348
CE842C3AA05FDBF0400892F03B5290FF5CA7E01C174FCEBC0B65012FF9F0AB1C19F483570599F9C6810
01F46C342CD6A491B2B4619FC70D30FF0CEAB49F2B5201ADCE25C510027496A4A64EA3A2661D55DA89
7416009769F2F501CE864F8360EC8EB7558CB24AAC577E338EA3DB515DA5199D159688C0A624D0C548
135A6001D2543D7274BEBA6C2AB648042C08CDF4732CB63FB0B57DDD967E8F577427EF19D669E0A16A
F77CA605491C1657744A1395065C1A459232A12743E62DEF8B6FC9F6075C8DA7E272CAE6874698FAD36
7A9C23CB28FDC001AE5501693FB5435BD871DDCC5BB0E236867F60AA04E52A7582DB8F25D0FC4CFD7
C4D223D67DADF248E0C920F0BCA7C9E774604FAEB56FAE025091FA595A7797C4331D3FCF9CEB881A13
92F231A4F05F4A4FFA58099802A20F7345EC7120BCE82ACD7C536B2627E4DD22F2CE177C27F38B544ED
991EF83A1F888A6217B48511E6A1FCFEF065FD84C459E4695E39AD5BB078B42751A798B780258E64B2D
A311A782D3B6FCD987515E870A54465F7C5302660B6F3A0CF88908EDE72743FBD55741B8D7B9C6BBAE
1A9D6F174DC4A9C52EB8129A8DE69F49A340BA65AF128E935FED44D57906B274905E6297C00358FBEE4
01742C5812C9BE54517770F48143B9E8E9EA39617834968AAC4A1494E3D5CF3B69BD5D1ED0392CF84B

```

86C64E4B29CDCB81C7FEA1ADEA09B9C27D6F31A5DB9EE6A27F1E0914D2180D5A6247A5BE6F2119FA13
919DC696B0C4EA95E5D96F850CB6D32B05F5505F48E2354F4953A2D2897F929D346ECC39F816DBF407
9893333A44AFA473B4A86FF96773295B48BA192FAAFEF79126FE0E944E1B1CE0D09C2BC745F62C237C3
E57EC61209ACF426E8FAB1DB3F8DDDE7AC6374866964178

Sample Decrypted Response

```
{
  "payInstrument": {
    "merchDetails": {
      "merchId": 8952,
      "merchTxnId": "Test123450",
      "merchTxnDate": "2021-12-06 T11:40:18"
    },
    "payDetails": {
      "atomTxnId": 11000000174925,
      "prodDetails": [
        {
          "prodName": "NSD",
          "prodAmount": 100.0
        }
      ],
      "amount": 100.0,
      "surchargeAmount": 1.18,
      "totalAmount": 101.18,
      "custAccNo": "213232323",
      "clientCode": "1234",
      "txnCurrency": "INR",
      "signature":
      "2b12c8bfc0e3a8268eddb6f406bf4187d4d0a0064d0355446986511453922c27e38367a97fff85863d48c147a8218e9e2d5003ab121f6f61ce3914030c60caac",
      "txnInitDate": "2021-12-06 11:40:19",
      "txnCompleteDate": "2021-12-06 11:40:23"
    },
    "payModeSpecificData": {
      "subChannel": [
        "NB"
      ]
    }
  },
}
```

```

"bankDetails": {
  "otsBankId": 2001,
  "bankTxnId": "vYmy8VmFzxLF6A7uDx0H",
  "otsBankName": "Atom Bank"
},
"extras": {
  "udf1": "",
  "udf2": "",
  "udf3": "",
  "udf4": "",
  "udf5": "",
  "udf6": "",
  "udf7": "",
  "udf8": "",
  "udf9": "",
  "udf10": ""
},
"custDetails": {
  "custEmail": "testuser@nttdata.com",
  "custMobile": "9753864210",
  "billingInfo": {}
},
"responseDetails": {
  "statusCode": "OTS0000",
  "message": "SUCCESS",
  "description": "TRANSACTION IS SUCCESSFUL."
}
}
}

```

Possible status code values, messages and description that will be posted in the final transaction response are mentioned below:

CODE	MESSAGE	DESCRIPTION	DESCRIPTION
OTS0000	SUCCESS	TRANSACTION SUCCESSFUL	

OTS0601	FAILED	IN STAGE TWO TRANSACTION, ATOM TXN ID SHOULD NOT BE NULL.	
OTS0602	FAILED	INCORRECT SURCHARGE AMOUNT.	
OTS0603	FAILED	SUBCHANNEL CAN NOT BE EMPTY.	
OTS0604	FAILED	IF SUBCHANNEL IS NB THEN BANK ID SHOULD NOT BE EMPTY.	
OTS0605	FAILED	IF SUBCHANNEL IS CC or DC THEN CARD DATA SHOULD NOT BE EMPTY.	
OTS0606	FAILED	IF SUBCHANNEL IS UP THEN VPA SHOULD NOT BE EMPTY.	
OTS0607	FAILED	IF SUBCHANNEL IS EM THEN EMI DETAILS SHOULD NOT BE EMPTY.	
OTS0608	FAILED	IN STAGE TWO TRANSACTION, SUBCHANNEL CAN NOT BE EMPTY.	
OTS0609	FAILED	CARD DETAILS ARE MISSING.	
OTS0610	FAILED	INVALID EMI TENURE.	
OTS0611	FAILED	INVALID EMI BANK NAME.	
OTS0612	FAILED	SUMMATION OF AMOUNT & SURCHARGE AMOUNT SHOULD BE EQUAL TO TOTAL AMOUNT.	
OTS0613	FAILED	RESPONSE IS ALREADY AVAILABLE IN DB FOR ATOM TXN ID.	
OTS0614	FAILED	ATOM's AMOUNT MISMATCHED WITH BANK's AMOUNT.	
OTS0615	FAILED		ANY OF THE MANDATORY PARAMETER MUST BE MISSING
OTS0616	FAILED	INSUFFICIENT MERCHANT INFORMATION.	
OTS0617	FAILED	IN STAGE TWO TRANSACTION, DB STAGE SHOULD BE 1.	
OTS0618	FAILED	IN STAGE TWO TRANSACTION, PROVIDED AMOUNT SHOULD BE EQUAL TO DB AMOUNT.	
OTS0619	FAILED	SUBCHANNEL & CARD TYPE MISMATCHED.	
OTS0620	FAILED	CARD SCHEME MISMATCHED.	
OTS0621	FAILED	AMOUNT SHOULD BE WITHIN A RANGE.	
OTS0622	FAILED	ACCOUNT DETAILS MANDATORY FOR PENNYDROP.	
		IN SEAMLESS TRANSACTION, WHEN MERCHANT IS ON	

OTS0623	FAILED	SURCHARGE THEN SURCHARGE AMOUNT SHOULD	
OTS0624	FAILED	TRANSACTION RESPONSE IS ALREADY AVAILABLE FOR ATOM TXN ID.	
OTS0625	FAILED	BANK TXN AMOUNT IS MISMATCHED WITH ATOM TXN AMOUNT.	
OTS0626	FAILED	AMOUNT & TOTAL AMOUNT OF ALL PASSED PRODUCTS SHOULD BE EQUAL.	
OTS0627	FAILED	INAPPROPRIATE WAY TO CONSUME OTS PAYMENT SERVICE.	
OTS0628	FAILED	ATOM TXN ID SHOULD NOT BE EMPTY/NULL IN BANK RESPONSE.	
OTS0629	FAILED	IN SEMI-SEAMLESS TRANSACTION, IF IT IS CARD TRANSACTION THEN SURCHARGE AMOUNT SHOULD NOT BE NULL/EMPTY/ZERO.	
OTS0630	FAILED	CARD TYPE MISMATCHED	
OTS0631	FAILED	FAILED-INVALID CARD NUMBER.	
OTS0632	FAILED	FAILED-CARD EXP MONTH/YEAR SHOULD BE IN PROPER FORMAT.	
OTS0633	FAILED	FAILED-CARD IS EXPIRED.	
OTS0634	FAILED	FAILED-INVALID CVV.	
OTS0635	FAILED	FAILED-TRANSACTION MODE NOT FOUND IN URL PATTERN.	
OTS0636	FAILED	FAILED-UNABLE TO DECRYPT CARD DETAILS	
OTS0637	FAILED	FAILED-UNABLE TO DECRYPT ENC DATA	
OTS0638	FAILED		ANY OF THE MANDATORY PARAMETER MUST BE MISSING FROM ENCDATA
OTS0639	FAILED	FAILED-FUTURE DATE NOT ALLOWED.	
OTS0640	FAILED	FAILED-DATE FORMAT MUST BE "+OTSUtil.ATOM_DATE_PATTERN";	
OTS0641	FAILED	MERCHID OR ENCDATA MISSING.	

Signature Generation

- For above mentioned API's request and their response signature fields to be generated using the shared hashing code.
- Signature type is HMACSHA512.

String to be encrypted:

[merchId + atomTxnId + merchTxnId + totalAmount + txnStatusCode + subChannel + bankTxnId]

Sample Request

1191110000001234abc12310.00OTS0000NB11dsff232311

Generated Signature

fb0798ae1602bc0d395b7a12e8f08ab5d0ec64e90a2b9e7f09ed3db847ac86820edb1a0d557c51aec2e0166fae9 7d4aa966a6f0d8512fd454e4deb906959c1bf

Hash Keys

Below is the merchant Id, request & response hash key –

MerchId	reqHashKey	respHashKey
9135	KEY1234567234	KERESPY1234567234

Encryption Key

Below is the UAT merchant id, request & response encryption key –

MerchId	encReqKey	encResKey
9135	58BE879B7DD635698764745511C704AB	7813E3E5E93548B096675AC27FE2C850

Signature Generation Java Code

```
import java.io.PrintStream;

import java.io.UnsupportedEncodingException;

import java.security.InvalidKeyException;

import java.security.Key;

import java.security.NoSuchAlgorithmException;

import javax.crypto.Mac;

import javax.crypto.spec.SecretKeySpec;

public class AtomSignature {

    public static String generateSignature(String hashKey, String[] param) {

        String resp = null;

        StringBuilder sb = new StringBuilder();

        for (String s : param) {
```

```

        sb.append(s);
    }
    try {
        System.out.println("String =" + sb.toString());

        resp = byteToHexString(
            encodeWithHMACSHA2(sb.toString(), hashKey)
        );
    } catch (Exception e) {
        System.out.println("Unable to encodc value with key : " + hashKey + " and input : " +
            sb.toString());
    }
    e.printStackTrace();
    return resp;
}

private static byte[] encodeWithHMACSHA2(String text, String keyString)
    throws NoSuchAlgorithmException, InvalidKeyException, UnsupportedEncodingException {
    Key sk = new SecretKeySpec(keyString.getBytes("UTF-8"), "HMACSHA512");
    Mac mac = Mac.getInstance(sk.getAlgorithm());
    mac.init(sk);
    byte[] hmac = mac.doFinal(text.getBytes("UTF-8"));
    return hmac;
}

public static String byteToHexString(byte byData[]) {
    StringBuilder sb = new StringBuilder(byData.length * 2);
    for (int i = 0; i < byData.length; i++) {
        int v = byData[i] & 0xff;
        if (v < 16)
            sb.append('0');
        sb.append(Integer.toHexString(v));
    }
    return sb.toString();
}

```

}

Note*: In case of NEFT/RTGS challan transaction, merchant will receive real-time transaction status as "Pending" as 1st stage when the Challan is generated successfully.

Post the payment as per NEFT/RTGS challan and reconciliation from NDPS, response will be posted only on Call Back URL (Webhook URL) configured against the Merchant MID. Hence merchants must have a Call Back URL compulsorily configured against their respective MID to receive the final transaction response.

AES Encryption Logic

- Transaction Status (Requery) API's request and returned response are shared via AES-512 encryption.
- The following KEY are to be used for UAT:

MerchId	encResKey	encReqKey
9135	58BE879B7DD635698764745511C704AB	7813E3E5E93548B096675AC27FE2C850

AES Encryption Logic Code

```
import java.util.logging.Logger;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;

public class AtomEncryption {

    static Logger log = Logger.getLogger(AtomEncryption.class.getName());
    private static int pswdIterations = 65536;
    private static int keySize = 512;

    private static final byte[] ivBytes = {
        0, 1, 2, 3, 4, 5, 6, 7,
        8, 9, 10, 11, 12, 13, 14, 15
    };

    public static String encrypt(String plainText, String key) {
        try {
            byte[] saltBytes = key.getBytes("UTF-8");

            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA512");
            PBEKeySpec spec = new PBEKeySpec(
                key.toCharArray(),
                saltBytes,
                pswdIterations,
                keySize
            );
```

```

        SecretKey secretKey = factory.generateSecret(spec);
        SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

        IvParameterSpec locallyIvParameterSpec = new IvParameterSpec(ivBytes);

        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(1, secret, locallyIvParameterSpec);

        byte[] encryptedTextBytes = cipher.doFinal(plainText.getBytes("UTF-8"));
        return byteToHex(encryptedTextBytes);

    } catch (Exception e) {
        log.info("Exception while encrypting data:" + e.toString());
    }
    return null;
}

public static String decrypt(String encryptedText, String key) {
    try {
        byte[] saltBytes = key.getBytes("UTF-8");
        byte[] encryptedTextBytes = hex2ByteArray(encryptedText);

        SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA512");
        PBEKeySpec spec = new PBEKeySpec(
            key.toCharArray(),
            saltBytes,
            pswdIterations,
            keySize
        );

        SecretKey secretKey = factory.generateSecret(spec);
        SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

        IvParameterSpec locallyIvParameterSpec = new IvParameterSpec(ivBytes);

        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(2, secret, locallyIvParameterSpec);

        byte[] decryptedTextBytes = null;
        decryptedTextBytes = cipher.doFinal(encryptedTextBytes);

        return new String(decryptedTextBytes);

    } catch (Exception e) {
        log.info("Exception while decrypting data:" + e.toString());
    }
    return null;
}

```

```

private static String byteToHex(byte[] byData) {
    StringBuffer sb = new StringBuffer(byData.length * 2);

    for (int i = 0; i < byData.length; ++i) {
        int v = byData[i] & 0xFF;

        if (v < 16)
            sb.append('0');

        sb.append(Integer.toHexString(v));
    }

    return sb.toString().toUpperCase();
}

private static byte[] hex2ByteArray(String sHexData) {
    byte[] rawData = new byte[sHexData.length() / 2];

    for (int i = 0; i < rawData.length; ++i) {
        int index = i * 2;
        int v = Integer.parseInt(sHexData.substring(index, index + 2), 16);
        rawData[i] = (byte) v;
    }

    return rawData;
}

public static void main(String[] args) {
    try {
        String encryptedData = AtomEncryption.encrypt(
            "1235",
            "ASWKLSLLFS4sd4g4gsdg"
        );
        System.out.println("encryptedData : " + encryptedData);

    } catch (Exception e) {
        // TODO: handle exception
    }
}

```

9. UAT Environment Details

The UAT environment details are as follows:

UAT IP: 13.127.25.237

UAT server: The UAT server needs to be whitelisted at the merchant's end so that response can be posted.

Test Credentials

UAT Test Credentials		Remarks
Payment Domain URL	https://paynetzuat.atomtech.in/ots/aipay/auth	
merchId	317159	Alternatively, can use MIDs – 317156, 317157, 8952
password	Test@123	Password
product/prodName	NSE / AIPAY	Product Name
Request Key/Salt	A4476C2062FFA58980DC8F79EB6A799E	Encryption Key
Response Key/Salt	75AEF0FA1B94B3C10D4F5B268F757F11	Decryption Key
Request Hash Key	KEY123657234	Used for signature validation in request (if applicable)
Response Hash Key	KEYRESP123657234	Used for signature validation in response (if applicable)

Test Card Details

Test - Credit Card	
Card Number:	4012888888881880
Expiry:	12/23
CVV	123
Card Name:	Test

Test - Debit Card	
Card Number:	5555555555554440
Expiry:	12/24
CVV	123
Card Name:	Test

Test UPI ID

Test UPI ID		Important Note
UPI ID	atomots@upi	Reach out prior to conducting transaction
UPI Name	Atom OTS Vpa	Auto populated once the UPI ID is verified

10. Sample Plain Request & Response (Single Account Settlement) for Multiple Payment Modes

Plain Request In JSON

```
{  
  "payInstrument": {  
    "headDetails": {
```

```

    "version": "OTSv1.1",
    "api": "AUTH",
    "platform": "FLASH"
  },
  "merchDetails": {
    "merchId": "317159",
    "userId": "",
    "password": "Test@123",
    "merchTxnDate": "2022-03-07 20:46:00",
    "merchTxnId": "test000123"
  },
  "payDetails": {
    "amount": "100",
    "product": "NSE",
    "custAccNo": "213232323",
    "txnCurrency": "INR"
  },
  "custDetails": {
    "custEmail": "user@atomtech.in",
    "custMobile": "9898989898"
  },
  "extras": {
    "udf1": "Merchant Data",
    "udf2": "Merchant Data",
    "udf3": "Merchant Data",
    "udf4": "Merchant Data",
    "udf5": "Merchant Data",
    "udf6": " Merchant Data ",
    "udf7": " Merchant Data ",
    "udf8": " Merchant Data ",
    "udf9": " Merchant Data ",
    "udf10": " Merchant Data "
  }
}

```

Net Banking Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:08:40"
    },
    "payDetails": {
      "atomTxnId": 11000000379742,
      "prodDetails": [
        {

```

```

        "prodName": "NSE",
        "prodAmount": 100
    }
],
    "amount": 100,
    "surchargeAmount": 0,
    "totalAmount": 100,
    "custAccNo": "213232323",
    "clientCode": "1234",
    "txnCurrency": "INR",
    "signature":
"1cbf87634568eabe245df5edf793f80b3de7796cc83246360cbb355a4b80cef2bc8f6a92245e213f6f03190298
72d2056d4422f650e6efc2b306aaa46f8d46a6",
    "txnInitDate": "2023-05-30 15:08:40",
    "txnCompleteDate": "2023-05-30 15:08:45"
},
    "payModeSpecificData": {
        "subChannel": [
            "NB"
        ],
        "bankDetails": {
            "otsBankId": 2001,
            "bankTxnId": "zNX8R7LZRZFMXba8YrRz",
            "otsBankName": "Atom Bank"
        }
    },
    "extras": {
        "udf1": "Merchant Data",
        "udf2": "Merchant Data",
        "udf3": "Merchant Data",
        "udf4": "Merchant Data",
        "udf5": "Merchant Data",
        "udf6": " Merchant Data ",
        "udf7": " Merchant Data ",
        "udf8": " Merchant Data ",
        "udf9": " Merchant Data ",
        "udf10": " Merchant Data "
    },
    "custDetails": {
        "custEmail": "user@atomtech.in",
        "custMobile": "9898989898",
        "billingInfo": {}
    },
    "responseDetails": {
        "statusCode": "OTS0000",
        "message": "SUCCESS",
        "description": "TRANSACTION IS SUCCESSFUL."
    }
}

```

```
}
```

Credit Card Sample Response in JSON

```
{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:07:50"
    },
    "payDetails": {
      "atomTxnId": 11000000379740,
      "prodDetails": [
        {
          "prodName": "NSE",
          "prodAmount": 100
        }
      ],
      "amount": 100,
      "surchargeAmount": 0,
      "totalAmount": 100,
      "custAccNo": "213232323",
      "clientCode": "1234",
      "txnCurrency": "INR",
      "signature":
"aa05829a4f42689e4488d75a84f0f5658cd4edc5d2a33772e1d971f28f5d4b8fb5848a07de8c0a1754b92b7d0
bcc44e3765afc664b197131394fff5271e22317",
      "txnInitDate": "2023-05-30 15:07:53",
      "txnCompleteDate": "2023-05-30 15:08:01"
    },
    "payModeSpecificData": {
      "subChannel": [
        "CC"
      ],
      "bankDetails": {
        "otsBankId": 2,
        "bankTxnId": "0011000000379740430",
        "authId": "00000",
        "otsBankName": "Hdfc Bank",
        "cardType": "CC",
        "cardMaskNumber": "XXXXXXXXXXXX1881",
        "scheme": "VISA"
      }
    },
    "extras": {
      "udf1": "Merchant Data",
      "udf2": "Merchant Data",
      "udf3": "Merchant Data",

```

```

    "udf4": "Merchant Data",
    "udf5": "Merchant Data",
    "udf6": " Merchant Data ",
    "udf7": " Merchant Data ",
    "udf8": " Merchant Data ",
    "udf9": " Merchant Data ",
    "udf10": " Merchant Data "
  },
  "custDetails": {
    "custEmail": "user@atomtech.in",
    "custMobile": "9898989898",
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "SUCCESS"
  }
}

```

Debit Card Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:11:21"
    },
    "payDetails": {
      "atomTxnId": 11000000379749,
      "prodDetails": [
        {
          "prodName": "NSE",
          "prodAmount": 100
        }
      ],
      "amount": 100,
      "surchargeAmount": 0,
      "totalAmount": 100,
      "custAccNo": "213232323",
      "clientCode": "1234",
      "txnCurrency": "INR",
      "signature":
        "c08f7e125a0979507aadb7c37a1ff829de694a1515361707960b4810fe3366cbd120e151af3ebacf3fb1c769b58f0559bee96f333f6dd62faacf36eb4b3ca082",
      "txnInitDate": "2023-05-30 15:11:23",
      "txnCompleteDate": "2023-05-30 15:11:31"
    }
  }
}

```

```

},
"payModeSpecificData": {
  "subChannel": [
    "DC"
  ],
  "bankDetails": {
    "otsBankId": 2,
    "bankTxnId": "0011000000379749430",
    "authId": "00000",
    "otsBankName": "Hdfc Bank",
    "cardType": "DC",
    "cardMaskNumber": "XXXXXXXXXXXX4444",
    "scheme": "VISA"
  }
},
"extras": {
  "udf1": "Merchant Data",
  "udf2": "Merchant Data",
  "udf3": "Merchant Data",
  "udf4": "Merchant Data",
  "udf5": "Merchant Data",
  "udf6": " Merchant Data ",
  "udf7": " Merchant Data ",
  "udf8": " Merchant Data ",
  "udf9": " Merchant Data ",
  "udf10": " Merchant Data "
},
"custDetails": {
  "custEmail": "user@atomtech.in",
  "custMobile": "9898989898",
  "billingInfo": {}
},
"responseDetails": {
  "statusCode": "OTS0000",
  "message": "SUCCESS",
  "description": "SUCCESS"
}
}
}

```

UPI Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:12:33"
    }
  },

```

```

"payDetails": {
  "atomTxnId": 11000000379751,
  "prodDetails": [
    {
      "prodName": "NSE",
      "prodAmount": 100
    }
  ],
  "amount": 100,
  "surchargeAmount": 0,
  "totalAmount": 100,
  "custAccNo": "213232323",
  "clientCode": "1234",
  "txnCurrency": "INR",
  "signature":
"f61affbc4c253aff7a3f85d8fd5d7ca44c46736ef0ea4595f3d924893ff07e7c70c6561b18347b5698aa62d9f300
8129fe391be9f80e6f73d073593b1bfd68c0",
  "txnInitDate": "2023-05-30 15:12:35",
  "txnCompleteDate": "2023-05-30 15:13:24"
},
"payModeSpecificData": {
  "subChannel": [
    "UP"
  ],
  "bankDetails": {
    "otsBankId": 2,
    "bankTxnId": "1685439751273",
    "authId": "2391685439751273",
    "otsBankName": "Hdfc Bank"
  }
},
"extras": {
  "udf1": "Merchant Data",
  "udf2": "Merchant Data",
  "udf3": "Merchant Data",
  "udf4": "Merchant Data",
  "udf5": "Merchant Data",
  "udf6": " Merchant Data ",
  "udf7": " Merchant Data ",
  "udf8": " Merchant Data ",
  "udf9": " Merchant Data ",
  "udf10": " Merchant Data "
},
"custDetails": {
  "custEmail": "user@atomtech.in",
  "custMobile": "9898989898",
  "billingInfo": {}
},
"responseDetails": {

```

```

    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "APPROVED OR COMPLETED SUCCESSFULLY"
  }
}

```

NEFT/RTGS Challan Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:14:02"
    },
    "payDetails": {
      "atomTxnId": 11000000379754,
      "prodDetails": [
        {
          "prodName": "NSE",
          "prodAmount": 100
        }
      ],
      "amount": 100,
      "surchargeAmount": 0,
      "totalAmount": 100,
      "custAccNo": "213232323",
      "clientCode": "1234",
      "txnCurrency": "INR",
      "signature":
"5252acc5b04a57a09e027d4986c8e2dc6dea46202e9c424e4555ad12ba40d3feb8333b34c96a9b1569670ed
71e41596c07a44a69e775d46eb42e53800c112ea9",
      "txnInitDate": "2023-05-30 15:14:03",
      "txnCompleteDate": ""
    },
    "payModeSpecificData": {
      "subChannel": [
        "NR"
      ],
      "bankDetails": {
        "otsBankId": 16,
        "bankTxnId": "ZATOMZ000000379754",
        "authId": "ZATOMZ000000379754"
      }
    },
    "extras": {
      "udf1": "Merchant Data",
      "udf2": "Merchant Data",

```

```

"udf3": "Merchant Data",
"udf4": "Merchant Data",
"udf5": "Merchant Data",
"udf6": " Merchant Data ",
"udf7": " Merchant Data ",
"udf8": " Merchant Data ",
"udf9": " Merchant Data ",
"udf10": " Merchant Data "
},
"custDetails": {
  "custEmail": "user@atomtech.in",
  "custMobile": "9898989898",
  "billingInfo": {}
},
"responseDetails": {
  "statusCode": "OTS0551",
  "message": "PENDING",
  "description": "CHALLAN GENERATED SUCCESSFULLY"
}
}
}

```

11. Sample Plain Request & Response (Split Settlement – Multi Product) for Multiple Payment Modes

Plain Request in JSON

```

{
  "payInstrument": {
    "headDetails": {
      "version": "OTSv1.1",
      "api": "AUTH",
      "platform": "FLASH"
    },
    "merchDetails": {
      "merchId": "317159",
      "userId": "",
      "password": "Test@123",
      "merchTxnDate": "2022-03-07 20:46:00",
      "merchTxnId": "test000123"
    },
    "payDetails": {
      "amount": "100.00",
      "product": "AIPAY",
      "custAccNo": "213232323",
      "txnCurrency": "INR",
      "prodDetails": [

```

```

{
  "prodName": "AIPAY",
  "prodAmount": "20.00"
},
{
  "prodName": "NSE",
  "prodAmount": "80.00"
}
]
},
"custDetails": {
  "custEmail": "user@atomtech.in",
  "custMobile": "9898989898"
},
"extras": {
  "udf1": "Merchant Data",
  "udf2": "Merchant Data",
  "udf3": "Merchant Data",
  "udf4": "Merchant Data",
  "udf5": "Merchant Data",
  "udf6": " Merchant Data ",
  "udf7": " Merchant Data ",
  "udf8": " Merchant Data ",
  "udf9": " Merchant Data ",
  "udf10": " Merchant Data "
}
}
}

```

Net Banking Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:45:44"
    },
    "payDetails": {
      "atomTxnId": 11000000379816,
      "prodDetails": [
        {
          "prodName": "AIPAY",
          "prodAmount": 100
        }
      ],
      "amount": 100,
      "surchargeAmount": 0,
      "totalAmount": 100,

```

```

    "custAccNo": "213232323",
    "clientCode": "1234",
    "txnCurrency": "INR",
    "signature":
"0bfac7fd1fc27660beec679f17da434305b7b05bb0e63641b850c078d63cd82f45b258c65e4bac165be64b74
320f0583574efc488299b275592110bfb3dda785",
    "txnInitDate": "2023-05-30 15:45:44",
    "txnCompleteDate": "2023-05-30 15:45:50"
  },
  "payModeSpecificData": {
    "subChannel": [
      "NB"
    ],
    "bankDetails": {
      "otsBankId": 2001,
      "bankTxnId": "NgvleGMKpD7BcNPSZMYn",
      "otsBankName": "Atom Bank"
    }
  },
  "extras": {},
  "custDetails": {
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "TRANSACTION IS SUCCESSFUL."
  }
}

```

Credit Card Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:54:22"
    },
    "payDetails": {
      "atomTxnId": 11000000379836,
      "prodDetails": [
        {
          "prodName": "AIPAY",
          "prodAmount": 100
        }
      ],
      "amount": 100,

```

```

    "surchargeAmount": 0,
    "totalAmount": 100,
    "custAccNo": "213232323",
    "clientCode": "1234",
    "txnCurrency": "INR",
    "signature":
    "c2f352c8830cc039f51d1a0d916ef30fac73abce78c740bc8b37cd0f4c1459cbc35b3a9dc953c07c1496e40ae
    006a3f2f9d4dc53049788f13189fbbd404187f8",
    "txnInitDate": "2023-05-30 15:54:24",
    "txnCompleteDate": "2023-05-30 15:54:32"
  },
  "payModeSpecificData": {
    "subChannel": [
      "CC"
    ],
    "bankDetails": {
      "otsBankId": 2,
      "bankTxnId": "0011000000379836430",
      "authId": "00000",
      "otsBankName": "Hdfc Bank",
      "cardType": "CC",
      "cardMaskNumber": "XXXXXXXXXXXX1881",
      "scheme": "VISA"
    }
  },
  "extras": {},
  "custDetails": {
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "SUCCESS"
  }
}

```

Debit Card Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,
      "merchTxnId": "test000123",
      "merchTxnDate": "2023-05-30T15:56:54"
    },
    "payDetails": {
      "atomTxnId": 11000000379842,
      "prodDetails": [

```

```

{
  "prodName": "AIPAY",
  "prodAmount": 100
},
{
  "amount": 100,
  "surchargeAmount": 0,
  "totalAmount": 100,
  "custAccNo": "213232323",
  "clientCode": "1234",
  "txnCurrency": "INR",
  "signature":
"279e30ef2d212ba9a6c83544bc3533ef8b6747dad737a2070eb2643b8511c1a7a1000d09b11c6e9a2396794f
f114702d2aee0757aeefd689f20e5a23f8753d25",
  "txnInitDate": "2023-05-30 15:56:57",
  "txnCompleteDate": "2023-05-30 15:57:04"
},
{
  "payModeSpecificData": {
    "subChannel": [
      "DC"
    ],
    "bankDetails": {
      "otsBankId": 2,
      "bankTxnId": "0011000000379842430",
      "authId": "00000",
      "otsBankName": "Hdfc Bank",
      "cardType": "DC",
      "cardMaskNumber": "XXXXXXXXXXXX4444",
      "scheme": "VISA"
    }
  },
  "extras": {},
  "custDetails": {
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "SUCCESS"
  }
}

```

UPI Sample Response in JSON

```

{
  "payInstrument": {
    "merchDetails": {
      "merchId": 317159,

```

```

    "merchTxnId": "test000123",
    "merchTxnDate": "2023-05-30T15:58:25"
  },
  "payDetails": {
    "atomTxnId": 11000000379843,
    "prodDetails": [
      {
        "prodName": "AIPAY",
        "prodAmount": 100
      }
    ],
    "amount": 100,
    "surchargeAmount": 0,
    "totalAmount": 100,
    "custAccNo": "213232323",
    "clientCode": "1234",
    "txnCurrency": "INR",
    "signature":
"30ff1ac95e9ab010415211def21ca35c81c471d95a8b0dbab437b0bdd33d1822b9cea0d7720396352b26954
e1bf39a1f1cf37ed97909ed7a4fed169110ce357d",
    "txnInitDate": "2023-05-30 15:58:26",
    "txnCompleteDate": "2023-05-30 15:58:41"
  },
  "payModeSpecificData": {
    "subChannel": [
      "UP"
    ],
    "bankDetails": {
      "otsBankId": 2,
      "bankTxnId": "1685442502736",
      "authId": "7401685442502736",
      "otsBankName": "Hdfc Bank"
    }
  },
  "extras": {},
  "custDetails": {
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0000",
    "message": "SUCCESS",
    "description": "APPROVED OR COMPLETED SUCCESSFULLY"
  }
}

```

NEFT / RTGS Challan Response in JSON

```
{
```

```

"payInstrument": {
  "merchDetails": {
    "merchId": 317159,
    "merchTxnId": "test000123",
    "merchTxnDate": "2023-05-30T16:00:31"
  },
  "payDetails": {
    "atomTxnId": 11000000379847,
    "prodDetails": [
      {
        "prodName": "AIPAY",
        "prodAmount": 100
      }
    ],
    "amount": 100,
    "surchargeAmount": 0,
    "totalAmount": 100,
    "custAccNo": "213232323",
    "clientCode": "1234",
    "txnCurrency": "INR",
    "signature":
"2c1a2e694195129565cc73b1c6c5e13c30daa2c7e7e339a0011ccbde3b850da6ec540da0e1dcf6f6b68be3e1
b9ce37ecbebb3cdf6b04be2cbb2ef850627ee8b9",
    "txnInitDate": "2023-05-30 16:00:32",
    "txnCompleteDate": ""
  },
  "payModeSpecificData": {
    "subChannel": [
      "NR"
    ],
    "bankDetails": {
      "otsBankId": 16,
      "bankTxnId": "ZATOMZ000000379847",
      "authId": "ZATOMZ000000379847"
    }
  },
  "extras": {},
  "custDetails": {
    "billingInfo": {}
  },
  "responseDetails": {
    "statusCode": "OTS0551",
    "message": "PENDING",
    "description": "CHALLAN GENERATED SUCCESSFULLY"
  }
}
}

```